

Dictionaries

```
student = {"name": "Nick", "age": 23}
```

```
student["sex"] = "male"
```

```
student["age"] = 24
```

```
age = student["age"] # OK
```

```
sex = student["sex"] # KeyError
```

```
"sex" in student # is "sex" a key ?
```

Counting bases

Can you produce a dictionary with counts of each base in a DNA sequence? Like this:

```
counts = {"A":6, "G":3, "T":3, "C":5}
```

Start with this:

```
dna = "AGTACCGATACATAGCC"
```

```
counts = {"A":0, "G":0, "T":0, "C":0}
```

Counting bases

```
dna = "AGTACCGATACATAGCC"
```

```
counts = {"A":0, "G":0, "T":0, "C":0}
```

```
for b in dna:  
    counts[b] = counts[b] + 1
```

Counting bases



```
dna = "AGTACCGNATACATAGCC"  
  
counts = {"A":0, "G":0, "T":0, "C":0}  
  
for b in dna:  
    counts[b] = counts[b] + 1
```

What happens if we run the code now?

Keep this under your pillow

```
dna = "AGTACCGATACATAGCC"
```

```
counts = {}  
for b in dna:  
    if b not in counts:  
        counts[b] = 0  
    counts[b] += 1
```

Turn it into a function

```
def count_bases(dna):  
    counts = {}  
    for b in dna:  
        if b not in counts:  
            counts[b] = 0  
        counts[b] += 1  
    return counts
```

```
base_counts = count_bases("AGTTACATCC")
```

Make a data structure

```
names = [ 'seqA' , 'seqB' , 'seqC' ]  
sequences = [ 'AGTCGA' , 'CCTCGT' , 'ATTCGT' ]
```



```
{ 'seqC' : { 'A' : 1, 'C' : 1, 'T' : 3, 'G' : 1 },  
  'seqB' : { 'C' : 3, 'T' : 2, 'G' : 1 },  
  'seqA' : { 'A' : 2, 'C' : 1, 'T' : 1, 'G' : 2 } }
```

Make a data structure

```
def sequence_counts(names, sequences):  
    result = {}  
    # your code here...
```

Use count_bases!

```
    return result
```

```
names = ['seqA', 'seqB', 'seqC']  
sequences = ['AGTCGA', 'CCTCGT', 'ATTCGT']
```

```
stats = sequence_counts(names, sequences)
```

Solution

Solution

Extract data

Fill name_list with names of sequences with more than one **A**

```
{ 'seqC' : { 'A' : 1, 'C' : 1, 'T' : 3, 'G' : 1 },  
  'seqB' : { 'C' : 3, 'T' : 2, 'G' : 1 },  
  'seqA' : { 'A' : 2, 'C' : 1, 'T' : 1, 'G' : 2 } }
```

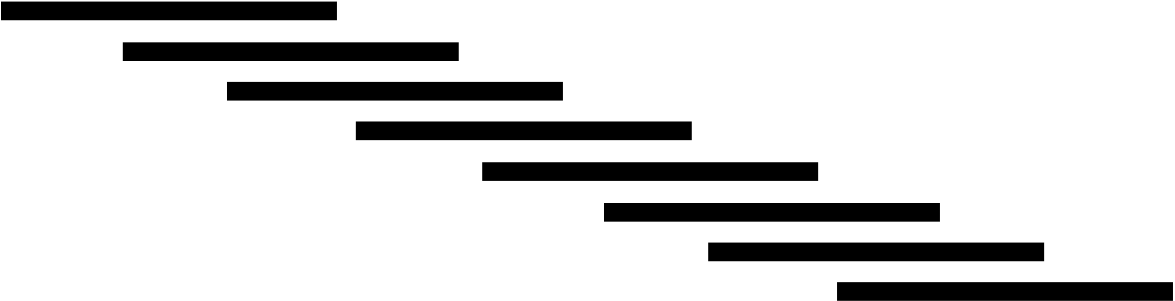
```
stats = sequence_counts(names, sequences)
```

```
name_list = []
```

Solution

k-mers

0 1 2 3 4 5 6 7 8 9
GAGAGAGCTA



3

`seq` = **'GAGAGAGCTA'**

Slicing

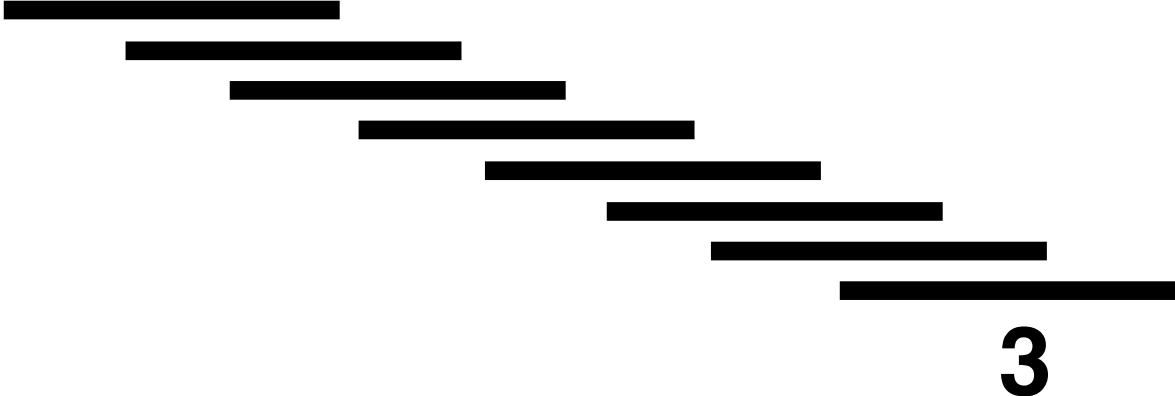
0 1 2 3 4 **5** 6 7 8 9
G A G A G **A G C** T A



3

```
green_kmer = seq[5:5+3]
```

0 1 2 3 4 5 6 7 8 9
GAGAGAGCTA



3

```
seq = 'GAGAGAGCTA'
```

```
k = 3
```

```
print(len(seq))
```

```
for i in range(????):
```

```
    print(i, seq[i:i+k])
```

What value should the last **i** be?

seq = 'GAGAGAGCTA'

Problem

Write a function: `count_kmers` that takes two arguments:

1. A string, which represents a DNA sequence.
2. An integer, which represent the length of kmers to count.

The function must return:

- A dictionary. Keys must be substrings of the string argument. The length of these substrings must be that given by the integer argument. The value for each key must be the number of times a substring appears in the string argument.

{ 'AGA' : 2, 'AGC' : 1, 'GCT' : 1, 'CTA' : 1, 'GAG' : 3 }

Problem

Write a function: `nr_different_kmers` that takes two arguments:

1. A string, which represents a DNA sequence.
2. An integer, which represent the length of kmers to count.

The function must return:

- An integer, which represent the number of different kmers in the string argument.

Problem

Write a function: `most_common_kmer` that takes two arguments:

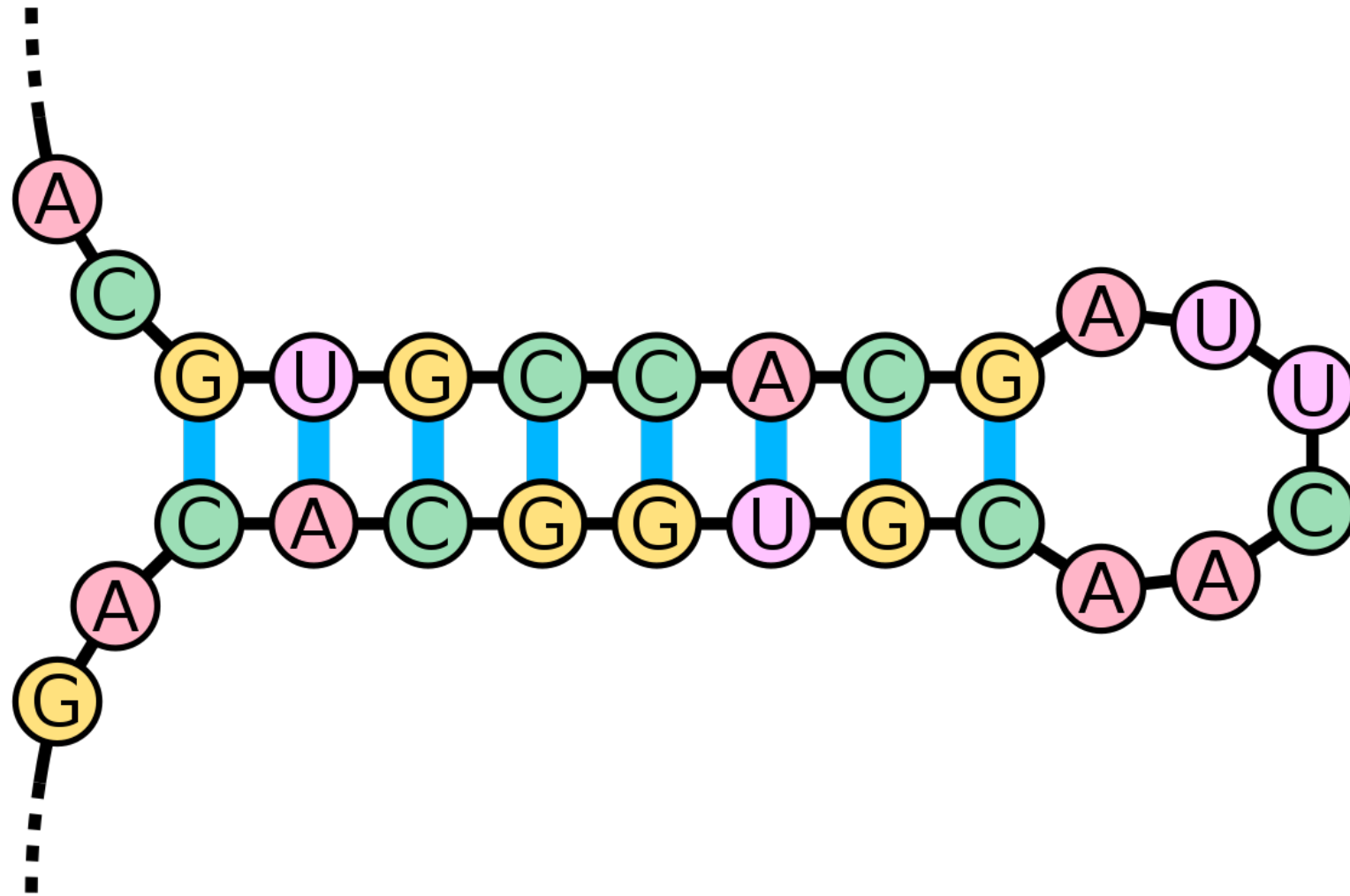
1. A string, which represents a DNA sequence.
2. An integer, which represent the length of kmers to count.

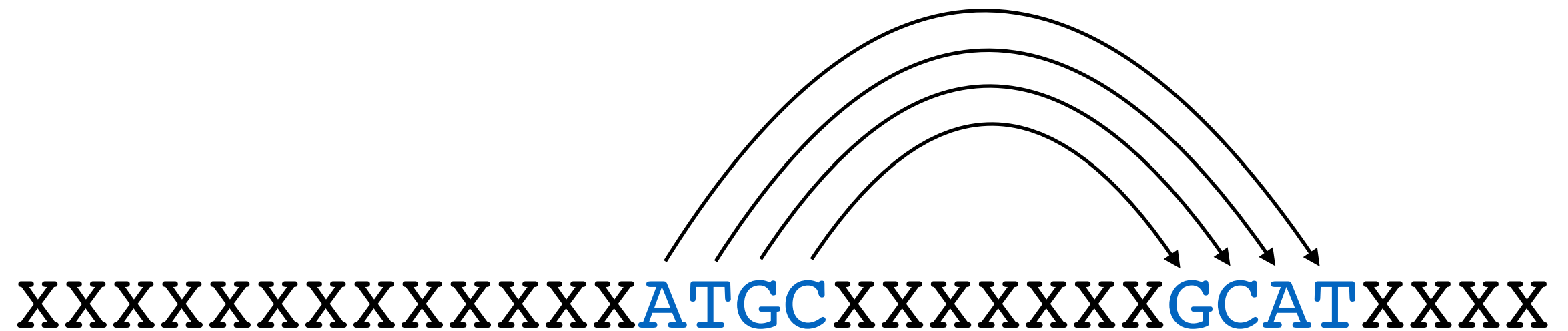
The function must return:

- An string, which represent the most common kmer in the string argument.

Primer design

Hairpin

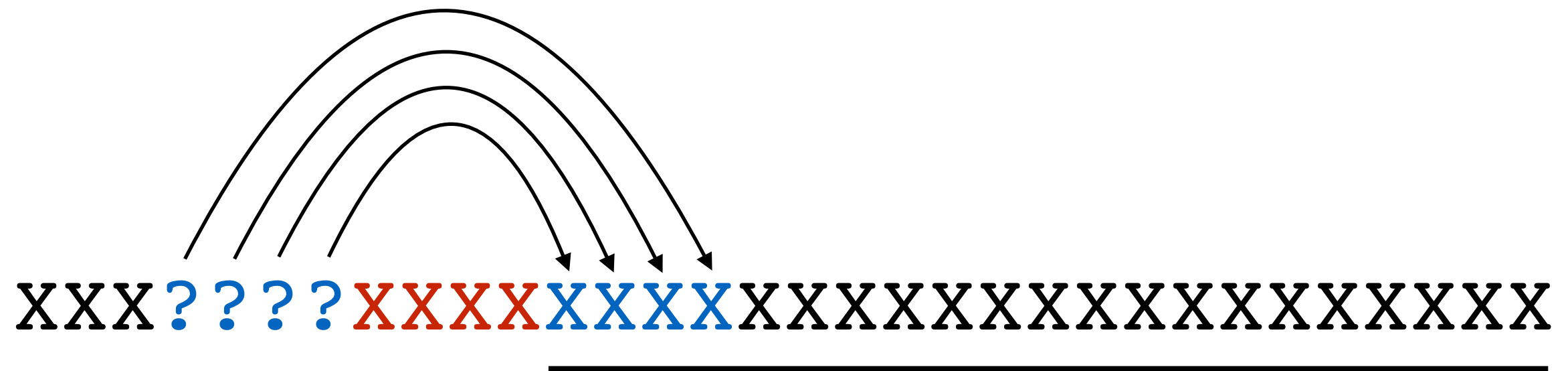




`GCAT` is reverse complement of `ATGC`

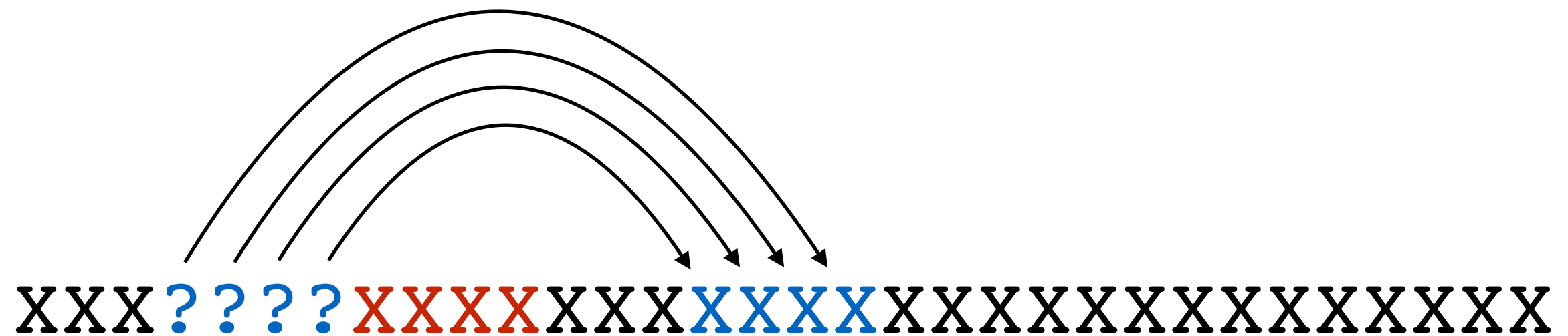
?? ? ? XX
X ? ? ? ? XX
XX ? ? ? ? XX
XXX ? ? ? ? XX
XXXXXXXX ? ? ? ? XX
XXXXXXXX ? ? ? ? XX
XXXXXXXX ? ? ? ? XX
...
XX ? ? ? ?

s[i:i+4]



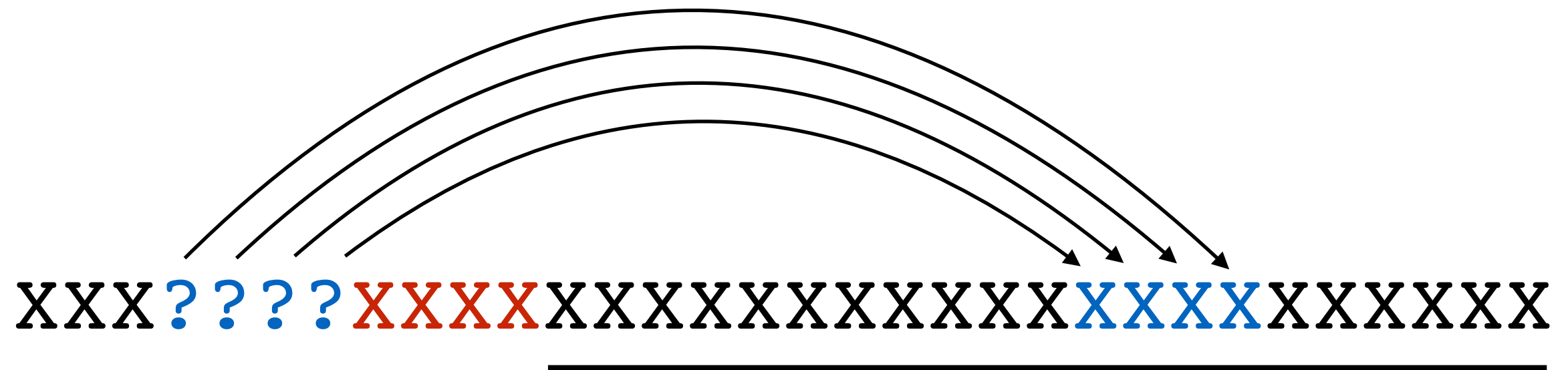
(Minimum loop size = four bases)

Reverse complement pairing to
any part of this sequence would
produce a hairpin loop



(Minimum loop size = four bases)

Reverse complement pairing to
any part of this sequence would
produce a hairpin loop



(Minimum loop size = four bases)

Reverse complement pairing to
any part of this sequence would
produce a hairpin loop

$$\mathbf{i} = 3$$
